

# Creare una REST API con PHP e JavaScript

## Lato PHP (server-side)

Come visto nelle lezioni precedenti, JSON è un linguaggio per formattare i dati che viene usato nei servizi Web come le REST API.

In PHP, è possibile utilizzare JSON per restituire dati all'utente che invia una richiesta alla nostra API. Supponiamo di voler implementare una semplice risposta della nostra API nel file `utente.php`:

```
<?php
header("Content-Type: application/json");

echo(json_encode([
    "id" => 1,
    "nome" => "Leonardo Essam",
    "cognome" => "Dei Rossi",
    "email" => "leonardo.deirossi@buonarroti.tn.it"
]));
?>
```

Lato JavaScript l'utente riceverà il seguente JSON:

```
{
  "id": 1,
  "nome": "Leonardo Essam",
  "cognome": "Dei Rossi",
  "email": "leonardo.deirossi@buonarroti.tn.it"
}
```

Osserviamo quindi che la funzione `json_encode(...)` prende come parametro un array associativo e lo converte in una stringa JSON. Per completezza, al contrario (per convertire da JSON ad un array associativo) vi è la funzione `json_decode(...)`.

## Lato JavaScript (client-side)

Per lavorare con le API in JavaScript si possono usare le chiamate AJAX (Asynchronous JavaScript and XML). Utilizzando la libreria jQuery (vedasi Classroom) si può definire una funzione per leggere i dati dell'utente:

```
async function leggiDatiUtente() {
  return new Promise((resolve, reject) => {
    $.ajax({
      method: "GET",
      url: "http://localhost/utente.php",
      headers: { /* Eventuale autenticazione */ },
      data: { /* Eventuali dati (es. nella POST) */ },
      success: (data) => {
        resolve(data);
      },
      error: (error, data) => {
        resolve(false);
      }
    });
  });
}
```

La funzione invia una richiesta asincrona al server e, in caso di successo (HTTP/X.Y 200 OK), restituisce i dati ricevuti, altrimenti `false`.

Per utilizzare tale funzione in un esempio:

```
let datiUtente = await leggiDatiUtente();
if (!datiUtente) {
    alert("Errore lettura dati utente!");
    return;
}

alert("Ciao, " + datiUtente.nome + "!");
```

Osserviamo l'uso della keyword `await`, ciò significa che il programma deve attendere che la funzione finisca la sua esecuzione (quindi che restituisca qualcosa) prima di proseguire con l'esecuzione del codice. In caso non venisse usata si rischia che si provi a utilizzare i dati dell'utente prima di scaricarli dal server!

**Per approfondire:** `jQuery.ajax()` ([link](#))